

CprE 381: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: Caden Otis and Christopher Hausner

Project Teams Group #: B_05

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

***We assume you mean the datapath values and control signals that are required and consumed in that stage - not including the ones that are propagated to later stages**

For the **IF** pipeline stage, it will not need any datapath values or control signals since we are just fetching the next instruction in the PC.

For the **ID** pipeline stage, we will need the following datapath values and control signals:

- s2_inst
- s_SignExtendImm

For the **EX** pipeline stage, we will need the following datapath values and control signals:

- s2_JumpSelect
- s2_JRSelect
- s2_Branch
- s2_BNE
- s2_OvflEN
- s2_ALUSrc
- s2_ShiftType
- s2_ALUControl
- s2_ReadData1
- s3_PC+4
- s2_DmemData
- s_Im32Bit

For the **MEM** pipeline stage, we will need the following datapath values and control signals:

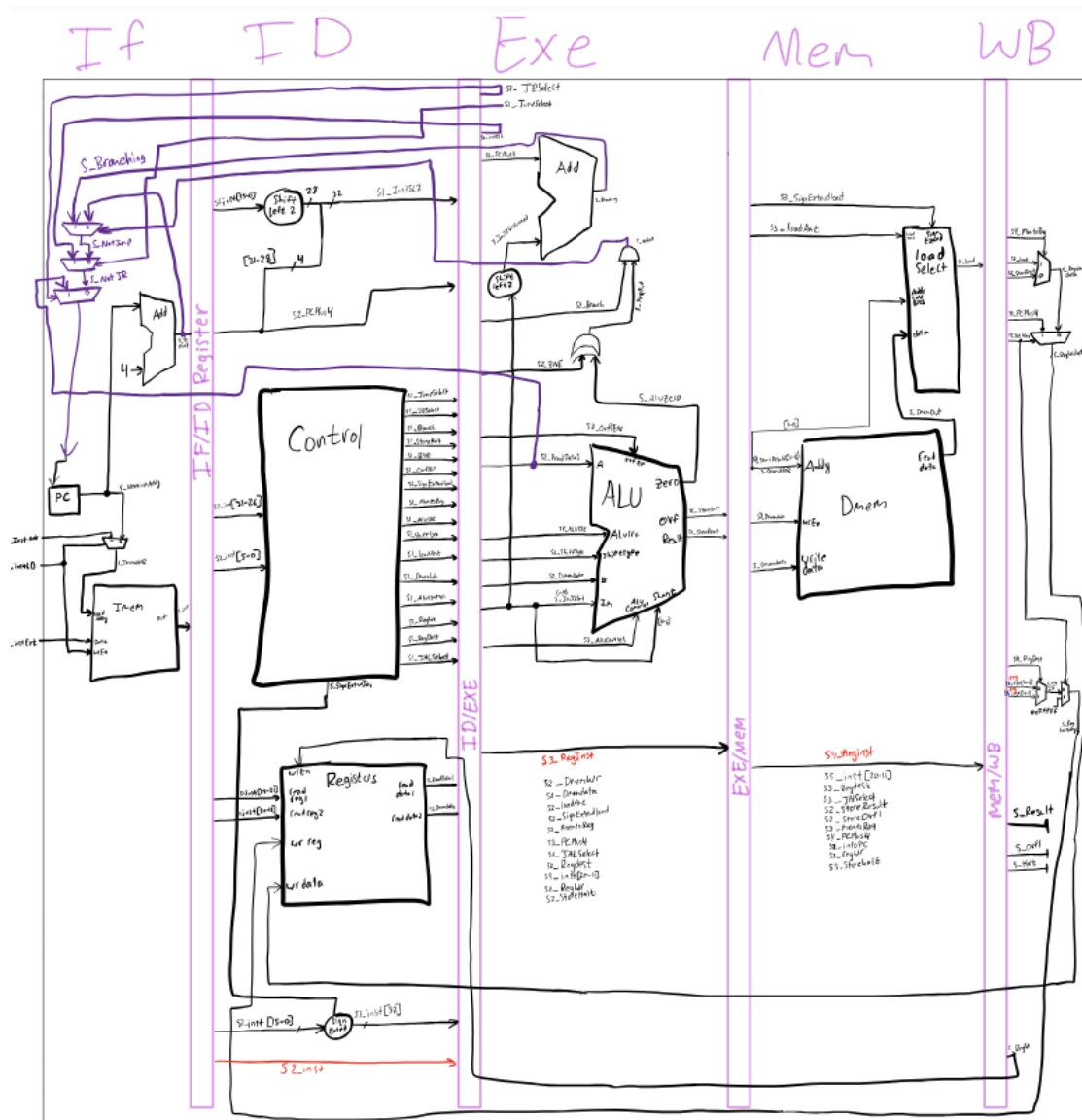
- s2_StoreResult
- s3_SignExtendLoad
- s3_LoadAmt
- s3_DmemWr
- s_DmemData

For the **WB** pipeline stage, we will need the following datapath values and control signals:

- s3_StoreResult
- s4_MemtoReg

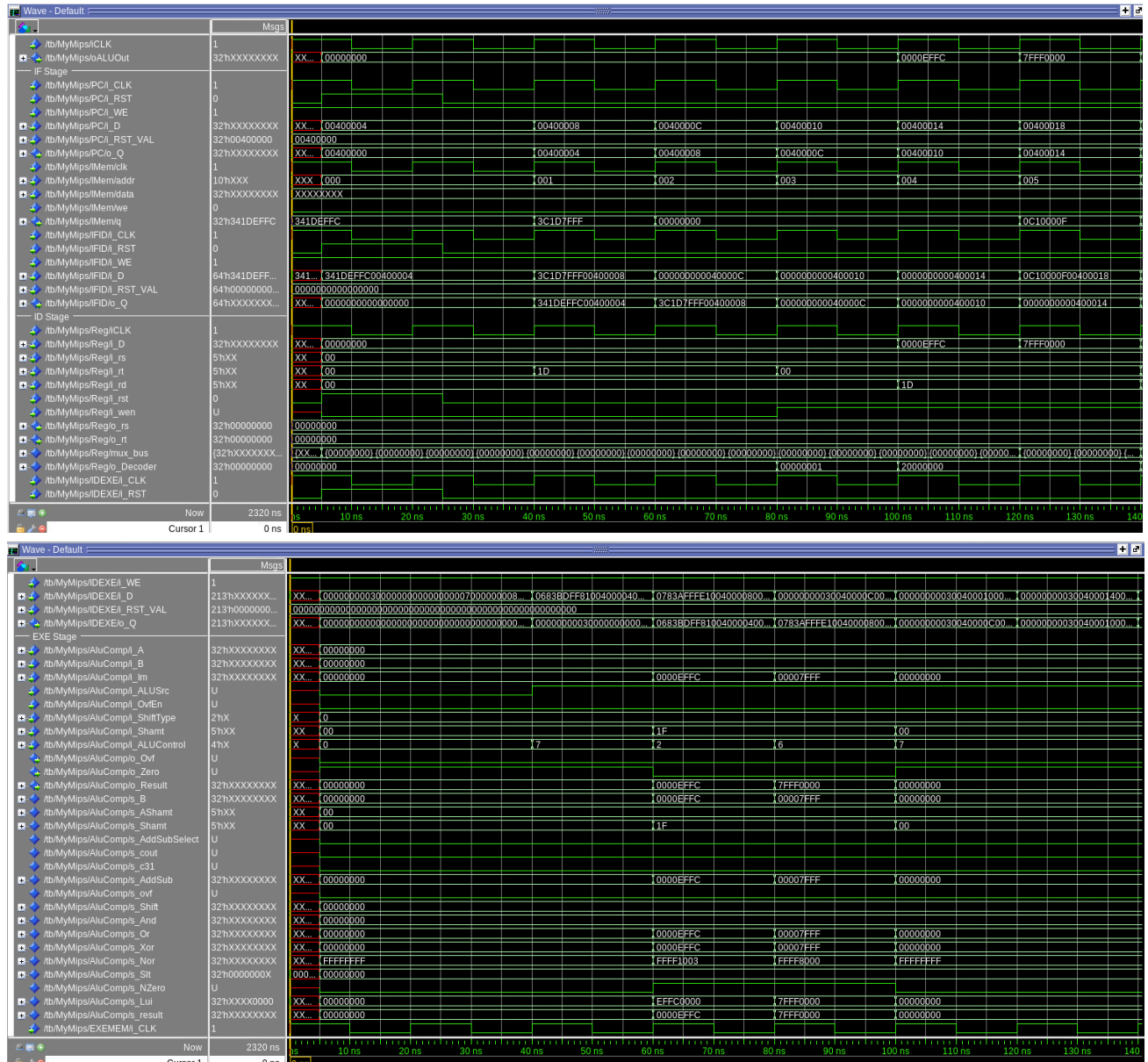
- s_RegWr
- s4_JALSelect
- s2_Load
- s5_PC+4
- s4_RegDest
- s5_RegInst
- s_Result
- s_Ovf
- s_Halt

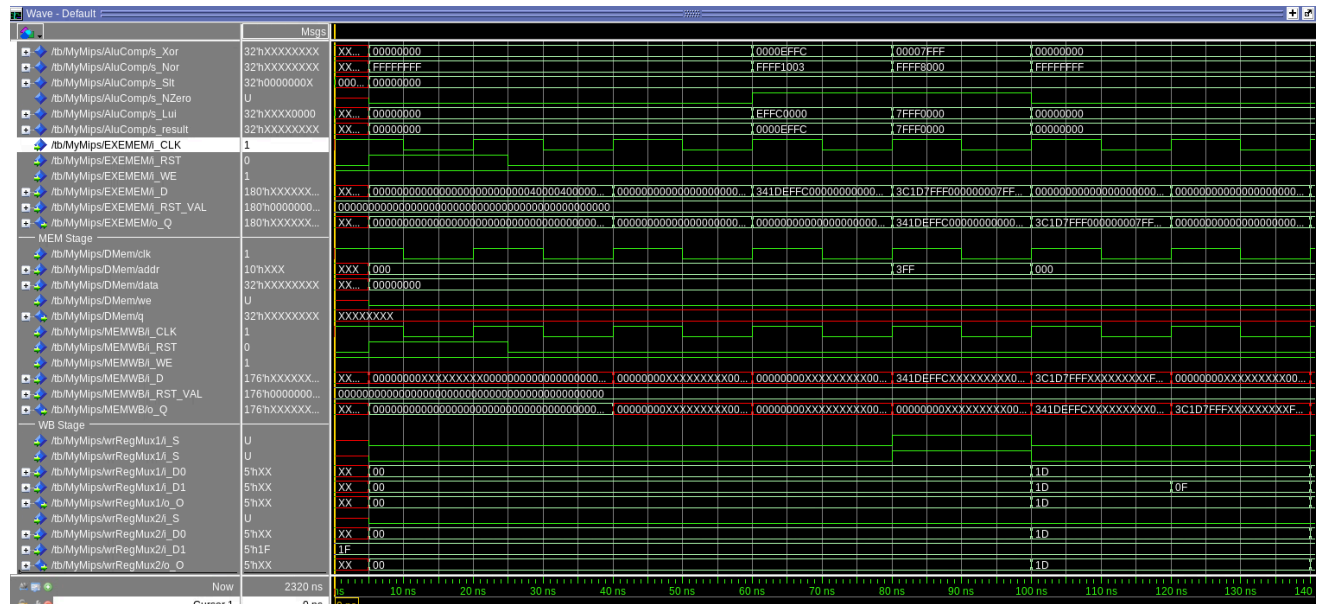
[1.b.ii] high-level schematic drawing of the interconnection between components.



[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.

In the test program that we wrote, there are quite a few occurrences of data hazards and control hazards that occur. To avoid these hazards, we use three nops (for data hazards) and four nops (for control hazards) to ensure that all register values have been written to the correct registers in time for when they are read, and all control flow instructions are stalled long enough to remove any incorrect instructions from the pipeline that aren't apart of the control flow taken. In the waveform screenshots below, it demonstrates the very first part of our test file, where three NOPs are executed to let the value 0x7FFFEFFC be written to the \$sp register before it creates an activation record on the stack.





The small program that we wrote was based on the Proj1_base_test.s file that we created for our last project. We modified this file to use every instruction that our processor can handle, and then we inserted NOP instructions in the code where data hazards (from read-after-write dependencies) or control hazards (from control flow instructions) would occur. In the waveform screenshots above. It demonstrates our processor fetching the first instruction to go into the IF stage then moving it along to the ID stage, while also getting the next instruction started in the IF stage. The final execution of this assembly file resulted in us having a **CPI of 1.16**. The purpose of making a pipelined processor was to get a CPI of about 1 to lower the execution time of applications run on the processor, and we were able to meet that with our design.

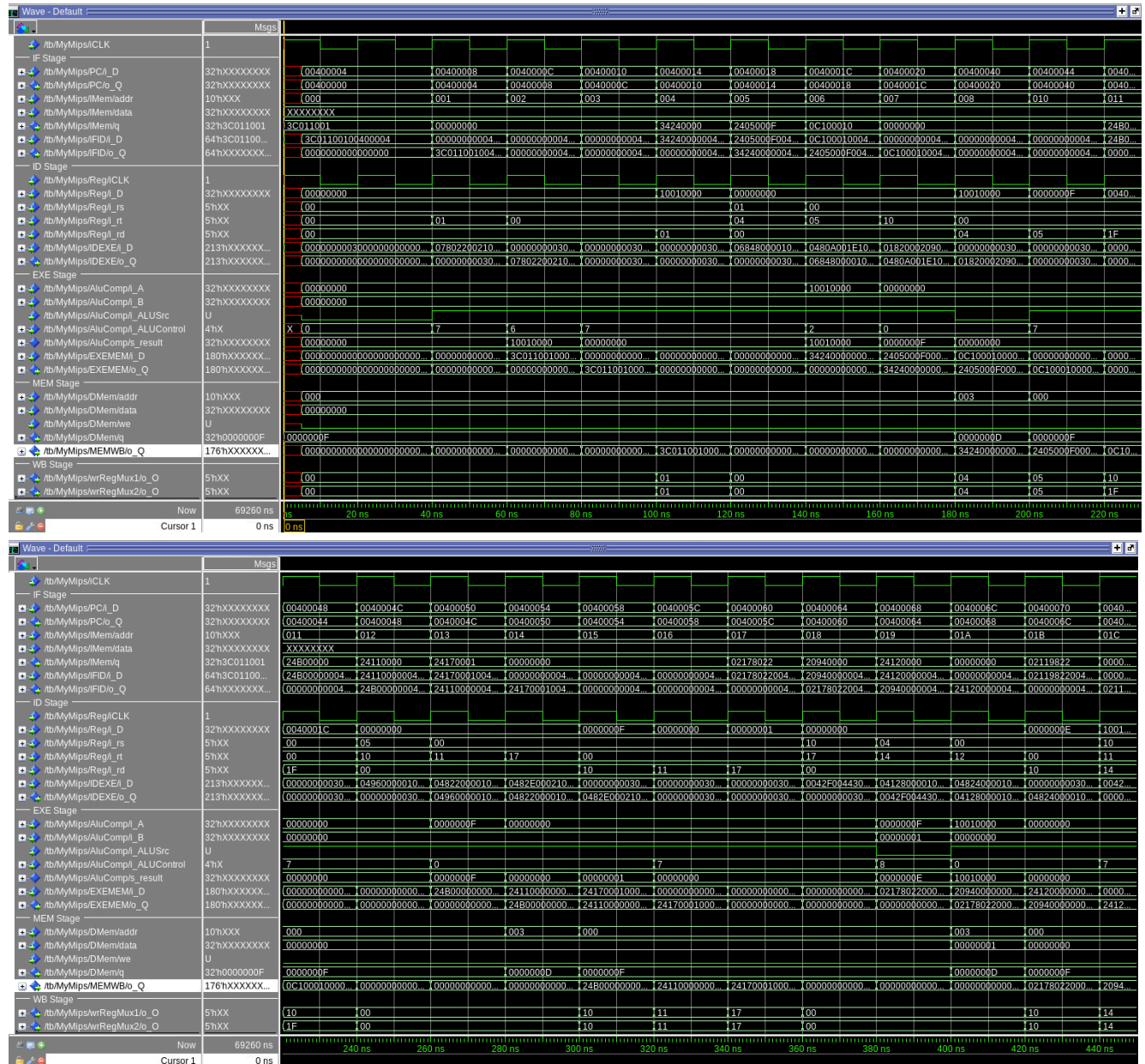
```

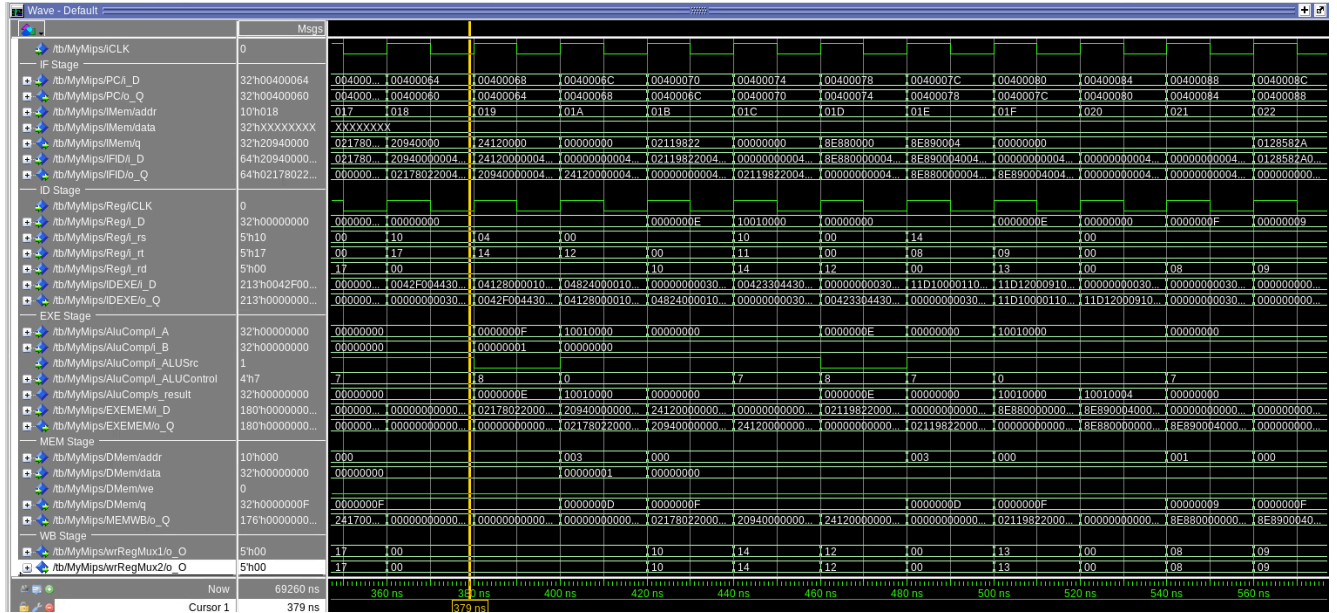
bash-4.4$ ls
381_tf.sh  cpre381-toolflow.pdf  internal  output  proj  temp
bash-4.4$ ./381_tf.sh test proj/mips/Proj2_base_test.s
Using VDI Python Environment
Testing
WARNING: Software tree location not set or invalid, using $MGC_HOME=/usr/local/m
entor/calibre
All VHDL src files compiled successfully
Testing file: proj/mips/Proj2_base_test.s
Mars simulation: pass
Modelsim simulation: pass
Test Result: pass
Mars Instructions: 99
Processor Cycles: 115
CPI: 1.16
Results in: output/Proj2 base test.s

```

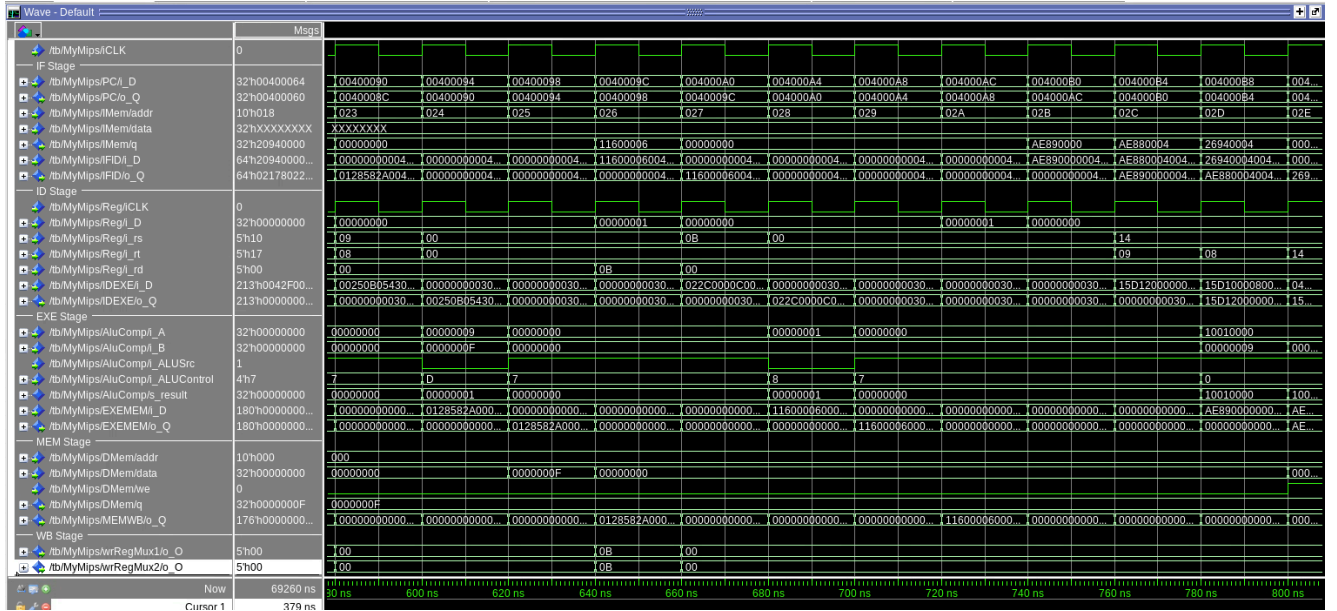
[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.

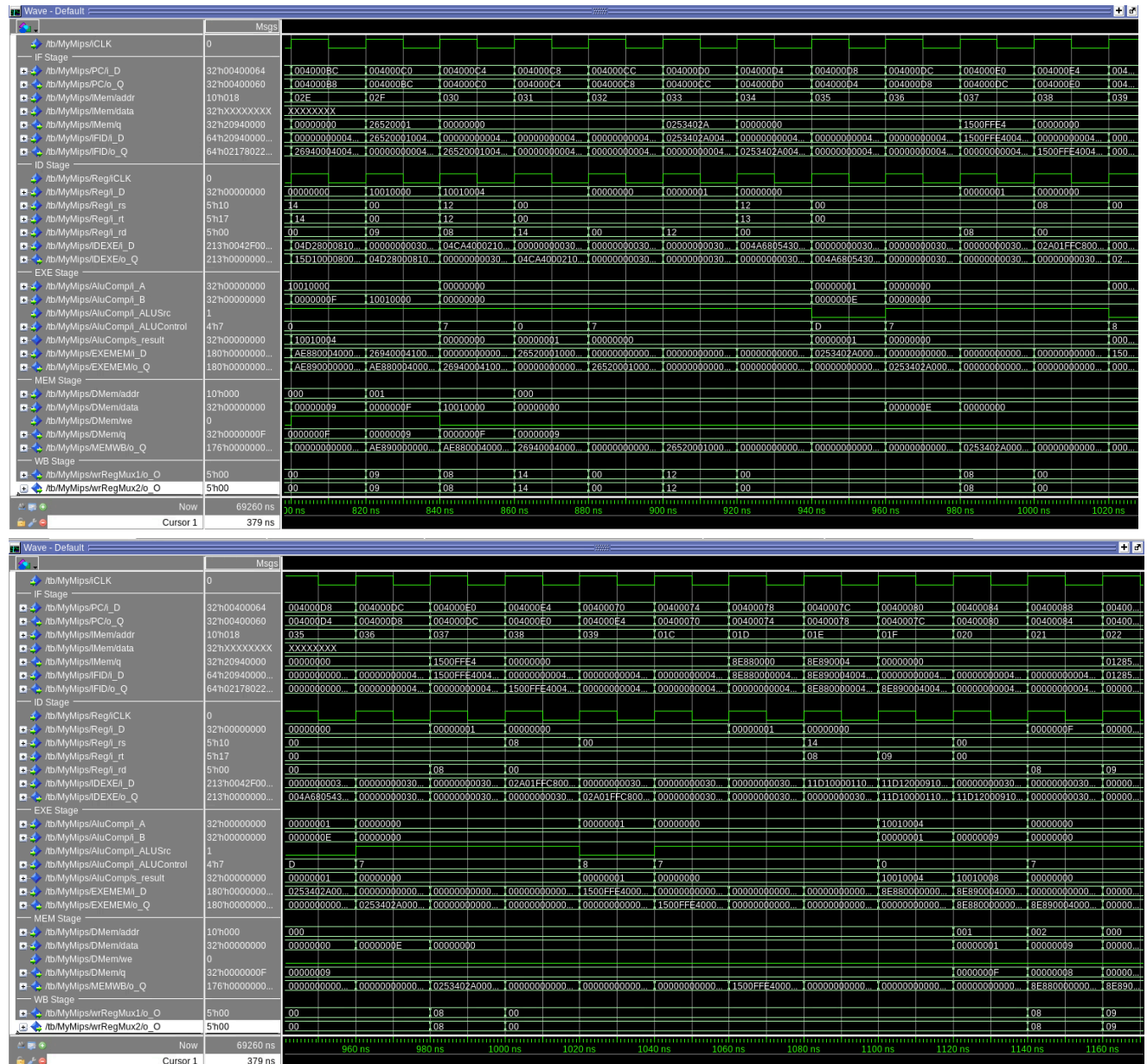
The below waveform screenshots are for the **very first iteration**:



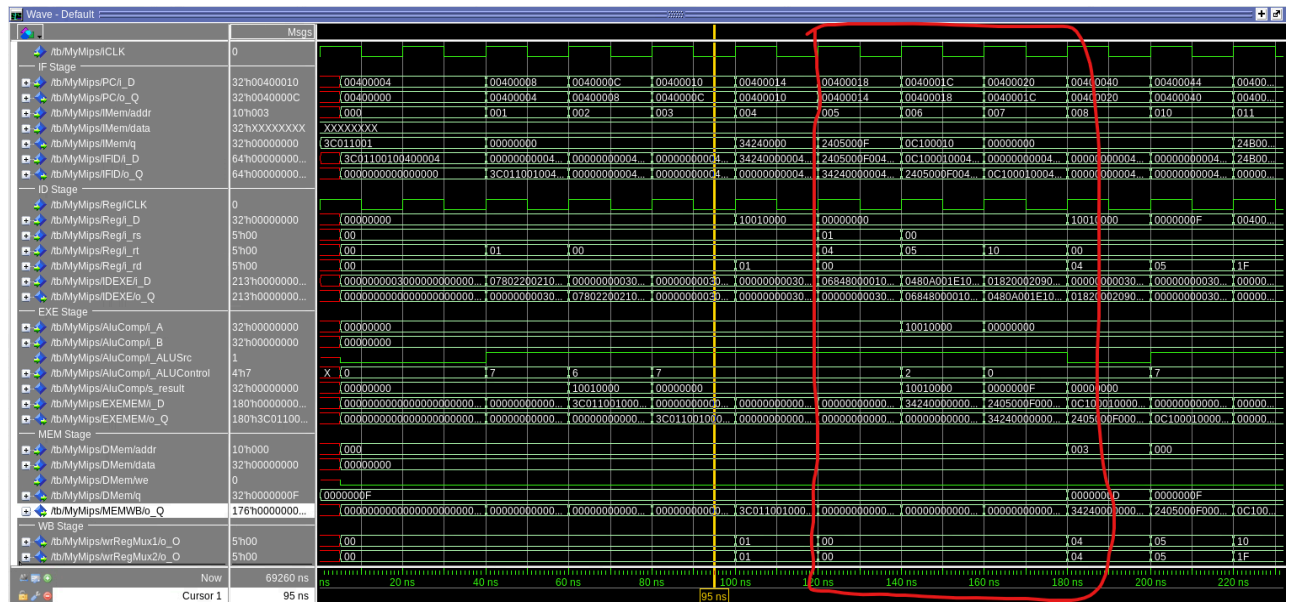


The below waveform screenshots are for the **second iteration** afterwards:



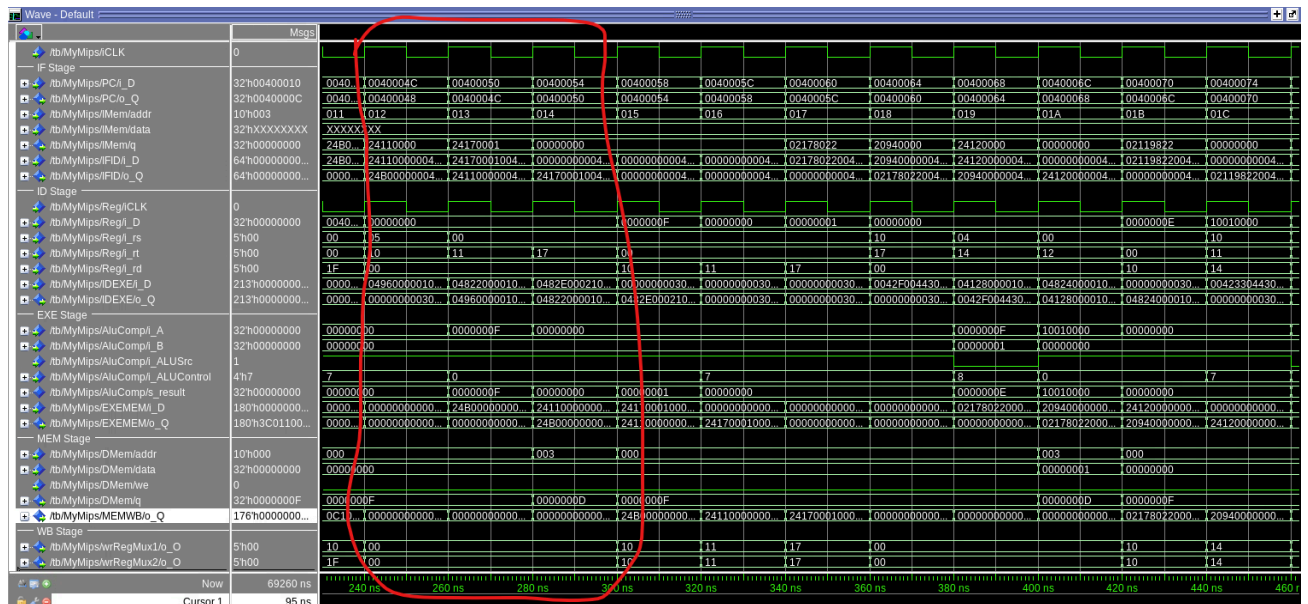


At the very beginning of our bubblesort algorithm, there is a read-after-write dependency for the register \$s7. As soon as it gets written to (addiu \$s7, \$0, 1), the next instruction tries to read from it (sub \$s0, \$s0, \$s7), which creates a data hazard by causing data flow to go backwards in time. To prevent this, we used **three NOP instructions** in between the two instructions so that the addiu instruction will be in the WB stage (new value is currently being written to \$s7) and the sub instruction is in the IF stage, and then the sub instruction can read the new value from \$s7 when it goes to the ID stage.



This example is from the first waveform screenshot in the first iteration of bubblesort

Once the algorithm enters the inner for loop, another data hazard occurs with the read-after-write dependency of \$t1. Once \$t1 has been written to (lw \$t1, 4(\$s4)), the next instruction tries to read that new value of \$t1 (slt \$t3, \$t1, \$t0), causing the data flow to go backwards in time. To prevent this data hazard from occurring, we put **three NOP instructions** in between the two instructions so that once the slt instruction enters the ID stage, the lw instruction will be exiting the pipeline with the new value having been written to \$t1 in the register file.



This example is from the second waveform screenshot in the first iteration of bubblesort

Near the end of the inner loop, we use control flow (the beq instruction) to decide whether two values within the array need to be swapped or not. If the beq instruction does branch to the NoSwap label, then a possible control hazard could exist. To prevent this, we use **four NOP**

instructions immediately after the branch to make sure data flow will continue to proceed forwards in time.



This example is from the third waveform screenshot in the first iteration of bubblesort

Once again, we achieved a CPI of about one when executing this test assembly file.

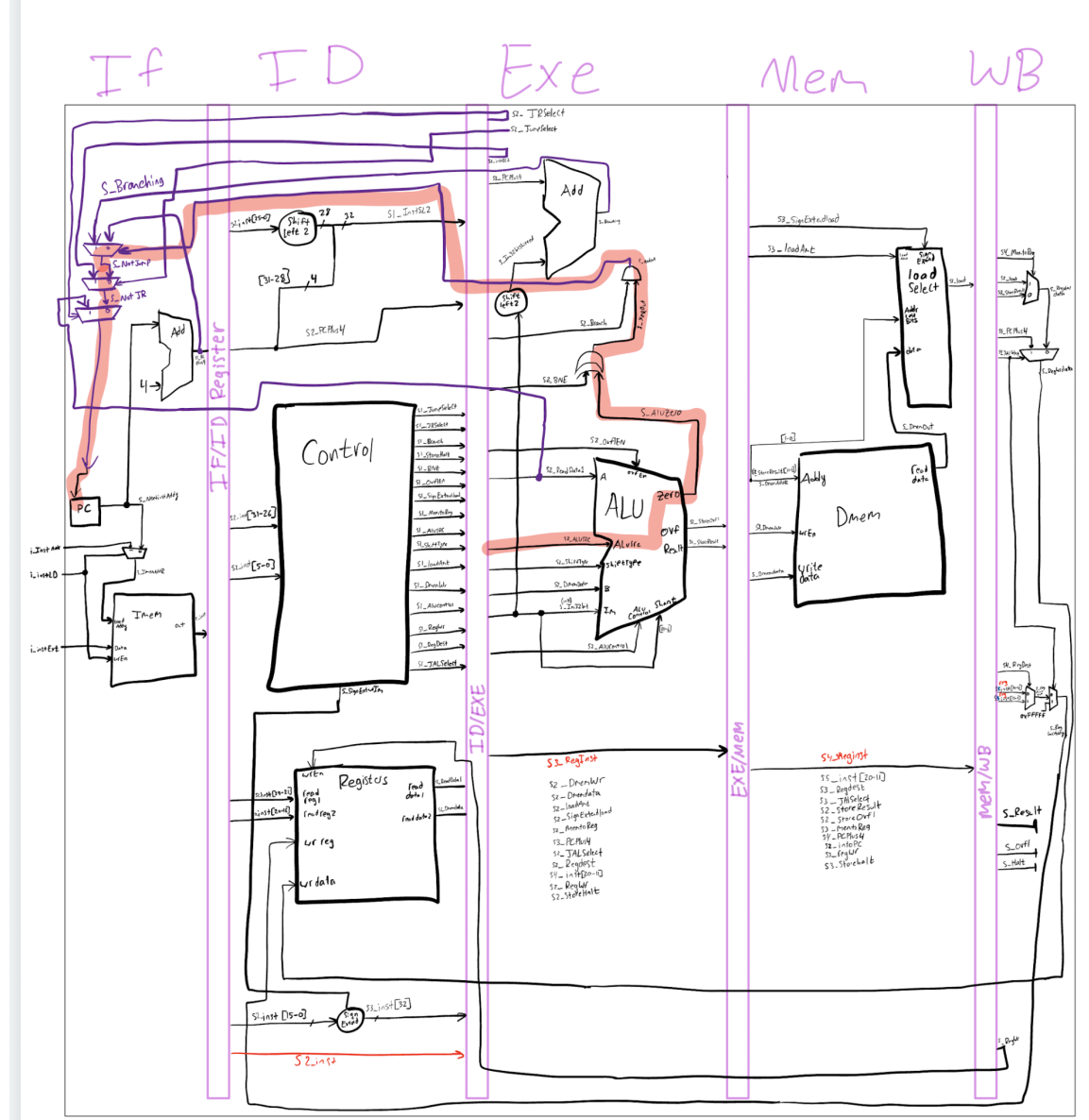
```
bash-4.4$ ./381_tf.sh test proj/mips/Proj2_bubblesort.s
Using VDI Python Environment
Testing
WARNING: Software tree location not set or invalid, using $MGC_HOME=/usr/local/mentor/calibre
All VHDL src files compiled successfully
Testing file: proj/mips/Proj2_bubblesort.s
Mars simulation: pass
Modelsim simulation: pass
Test Result: pass
Mars Instructions: 3134
Processor Cycles: 3462
CPI: 1.1
Results in: output/Proj2_bubblesort.s
```

[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

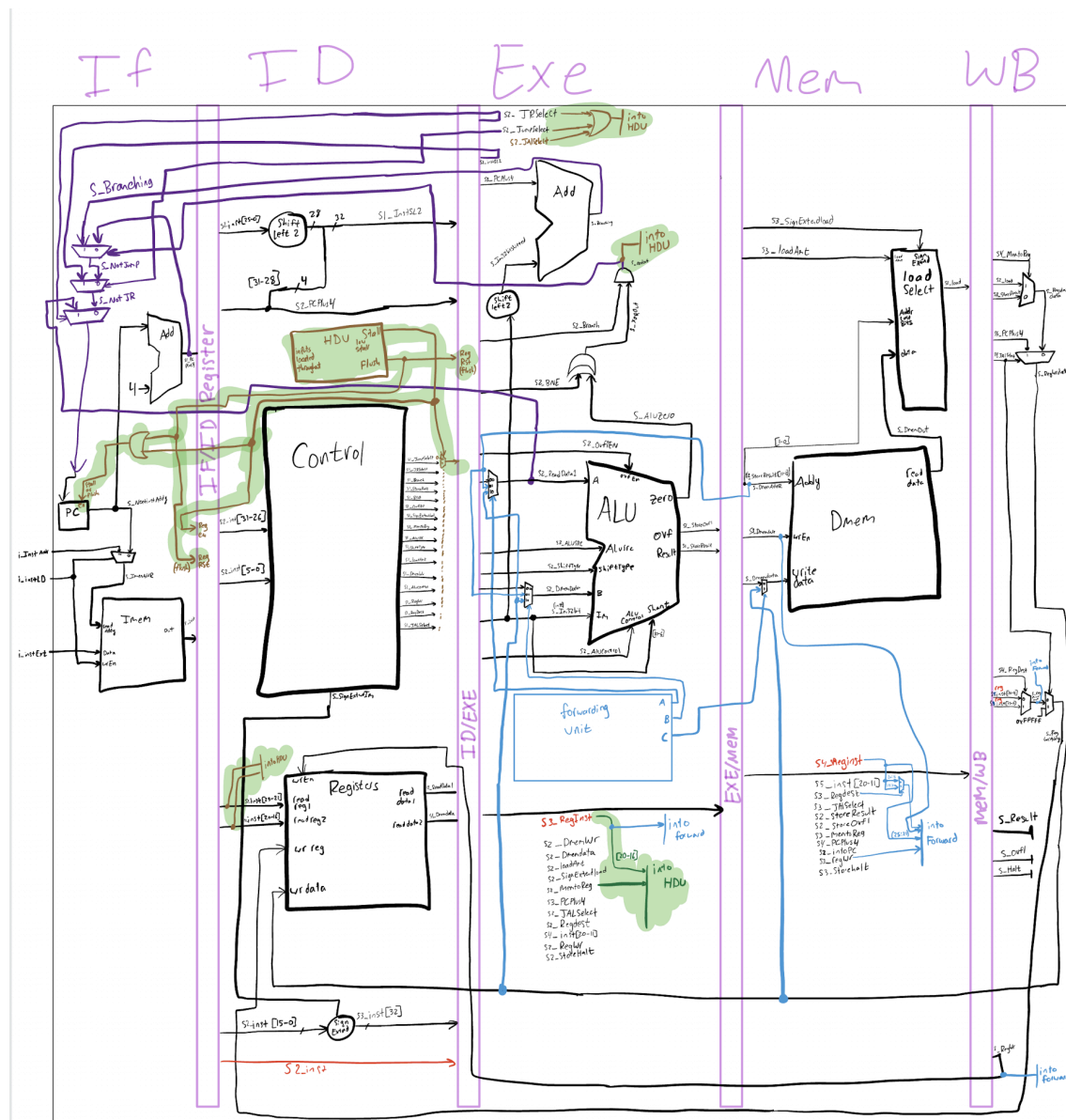
The maximum frequency that our software-scheduled pipelined processor can run at is **54.02 MHz**.

The critical path of our software-scheduled pipelined processor will first start at the ID/EXE pipeline register. It will then go through the ALUComponent. Inside of our ALU Component it will go through the adder subtractor. From the adder subtractor it will go through our control mux for the ALU set the zero flag. Then it goes to the BNE XOR gate and into the AND gate with s_branch. Once leaving the Branch register, our critical path will then go through three MUXes:

the BranchMux, the JumpMux, and the JRMux. Once going through those three MUXes, it will then go to the PC register. This is part of our EXE Stage, where we are doing address calculation.



[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.

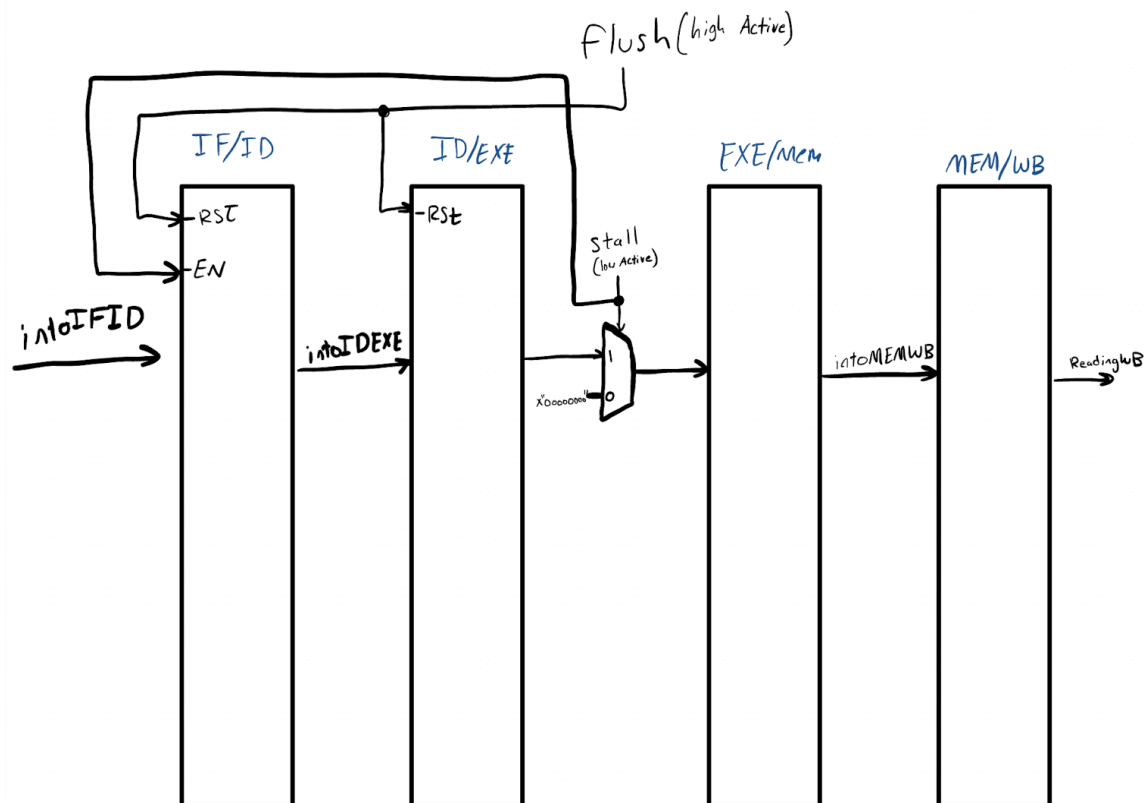


Here is a drawn schematic of how flushing and stalling work. Specifically, I will explain how we did our Flush and Stall signals. Flush is set to 1 when we are taking a branch or we are jumping. This flush is hooked up to ID/EXE Rst, to a OR gate for PCEN, and to IF/ID RST value. We changed our Registers to be Synchronous to avoid rst happening as a propagation delay. When we are flushing, PC EN = 1, which is good. However we are resetting our 2 registers of ID/EXE and IF/ID. By resetting we wont execute instructions after the branch or jump when we don't want to.

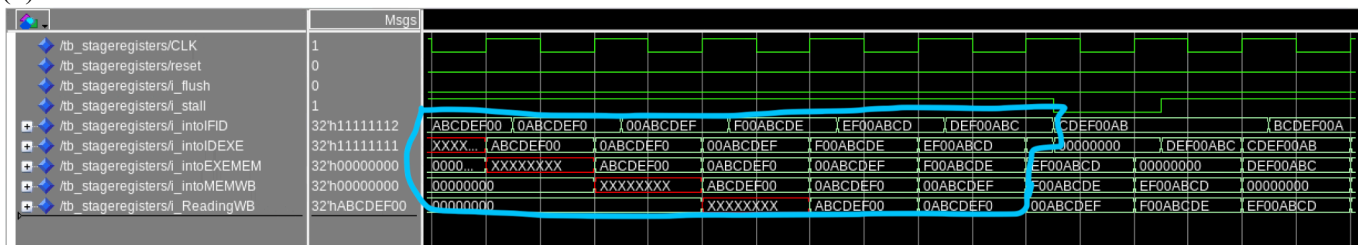
As for the Stalling, with forwarding there is only one stall case. That is when we are reading a register that was loaded into with a lb, lw, lbu, lh, lhu. For stalling, we know this is our case when EXE_RegWr & EXE_MemtoReg are both 1. But we need to check if we are actually using the values or not. Todo this we check both the ID_RS and ID_RT with EXE_RT. We OR that output, so if it matches either of them. All together if the RegWR&MemtoReg of EXE are 1 and also we are using the value, then stall is set to 0. We chose to do a low stall due to the fact that it can be hooked up directly to our registers. PC EN is only set to 0 when there is no flushing & stalling. In addition, for stalling our controls are set to 0 through a mux selector for stalling which will

implement NOPS and hold the PC where it is at. With this, stalling and flushing are implemented. (High flush, Low stall). Our HDU and Outputs/Inputs are highlighted above in the schematic in Green.

[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.



Above is a illustration of what I will simulate in my testbench. It follows the logic in my processor. We have 4 stage registers and we have a low stall and high flush. When stall is active (0) then we will use a mux and select 0x0 to move forward as our NOP.

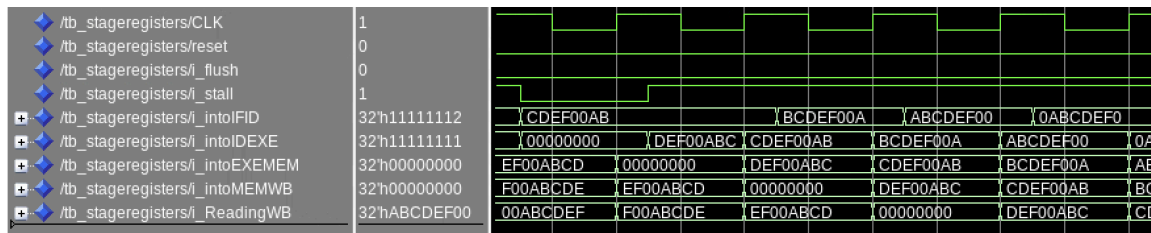


```
i_intoIFID <= x"ABCDEF00";
```

--Below shows that values trickle through the registers as we would expect.

```
wait for gCLK_HPER*2;
i_intoIFID <= x"0ABCDEF0";
wait for gCLK_HPER*2;
i_intoIFID <= x"00ABCDEF";
wait for gCLK_HPER*2;
i_intoIFID <= x"F00ABCDE";
wait for gCLK_HPER*2;
i_intoIFID <= x"EF00ABCD";
wait for gCLK_HPER*2;
i_intoIFID <= x"DEF00ABC";
wait for gCLK_HPER*2;
```

The images above shows that values are loaded into our registers one by one. Note that intoIFID is being written to not on a clock edge, as the testbench setup has that value not changing on clock edge. As u can see the values are loaded in as we would expect.

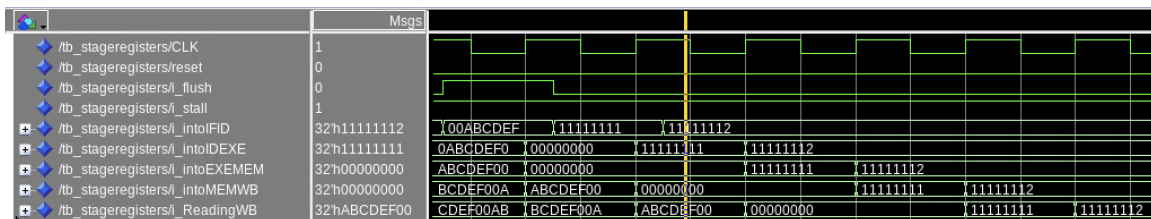


--Below is going to show how a stall would work.

```
i_stall <= '0'; --Low stall, would turn the PC off => Into IFID Stays the same.
i_intoIFID <= x"CDEF00AB";
wait for gCLK_HPER*2;
i_stall <= '1'; --Stall back off.
i_intoIFID <= x"CDEF00AB";
wait for gCLK_HPER*2;

i_intoIFID <= x"BCDEF00A";
wait for gCLK_HPER*2;
```

The images above show that when we stall (0=stall) then our EXEMEM becomes Zero, and that zero trickles down as we just performed one NOP and continued our programs after. And following our barrel shifting pattern we have been doing we see that the WB stage performs everything in order except for a NOP and then continues the order.



```

i_intoIFID <= x"ABCDEF00";--Now we will perform a Flush when this instruction is in the EXE Stage
wait for gCLK_HPER*2;

i_intoIFID <= x"0ABCDEF0";--ID Stage
wait for gCLK_HPER*2;
i_intoIFID <= x"00ABCDEF";--EXE STAGE
i_flush <= '1'; --Flush On.
wait for gCLK_HPER*2;
i_intoIFID <= x"11111111";
i_flush <= '0'; --Flush Off.
wait for gCLK_HPER*2;
i_intoIFID <= x"11111112";
wait for gCLK_HPER*2;

```

Our registers have synchronous reset - this way we can hook up the IRST value directly to flush. The images above show a flush in action. When we set flush to 1 (active) we see that on the next cycle the IF and the ID stage become 0's. We flushed the value in them. This flush takes 2 cycles as we would for a branch or jump and trickles down. then we continue with our new value I set it to of all 1's. This demonstrates that our flush and stall works correctly.

[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

The instructions that will produce values in the pipeline are: add, addi, addiu, addu, and, andi, lui, lw, nor, xor, xori, or, ori, slt, slti, sll, srl, sra, sub, subu, lb, jal, lbu, lhu, sllv, srlv, srav. Most instructions will produce their final values in the execute stage (output of the ALU) and they will be ready for forwarding at the beginning of the memory stage. So the following signals can be forwarded from the **s2_StoreResult** signal: add, addi, addiu, addu, and, andi, nor, xor, xori, or, ori, slt, slti, sll, srl, sra, sub, subu, lbu, lhu, sllv, srlv, srav

The instructions that use Dmem can be forwarded in the write back stage from the **s_RegWrData** signal: lui, lw, lb, jr, lbu, lhu

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

Most instructions will consume values in the execute stage (inputs of the ALU) and they will be ready for forwarding at the beginning of the memory stage. So the following signals can be forwarded from the **s2_ReadData1** signal: add, addi, addiu, addu, and, andi, lw, sw, nor, xor, xori, or, ori, slt, slti, sub, subu, lb, jr, lbu, lh, lhu, sllv, srlv, srav, beq, bne

Most instructions will consume values in the execute stage (inputs of the ALU) and they will be ready for forwarding at the beginning of the memory stage. So the following signals can be forwarded from the **s2_DmemData** signal: add, addi, addiu, addu, and, andi, sw, nor, xor, xori, or, ori, slt, sll, srl, sra, sub, subu, sllv, srlv, srav, beq, bne

There is also a consumer in Dmem for the signal **s_DmemData** that will have the following instruction: sw

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

The following instructions **can be forwarded with no issues**: add, addi, addiu, addu, and, andi, nor, xor, xori, or, ori, slt, slti, sll, srl, sra, sub, subu lbu, lhu, sllv, srlv, srav, beq, bne

The following instructions **can't be forwarded right away**, but will **need to be stalled at least once** first to then be able to forward the produced final value: lw, lb, lbu, lhu

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

***We assume you mean the datapath values and control signals that are required and consumed in that stage - not including the ones that are propagated to later stages**

For the **IF** pipeline stage, we will need no datapath values and the following control signals:

- s_Stall
- s_Flush

For the **ID** pipeline stage, we will need the following datapath values and control signals:

- s2_inst
- s_SignExtendmm
- s_Stall
- s_Flush

For the **EX** pipeline stage, we will need the following datapath values and control signals:

- s2_JumpSelect
- s2_JRSelect
- s2_Branch
- s2_BNE
- s2_OvflEN
- s2_ALUSrc
- s2_ShiftType
- s2_ALUControl
- s2_ReadData1
- s3_PC+4
- s2_DmemData
- s_Im32Bit

For the **MEM** pipeline stage, we will need the following datapath values and control signals:

- s2_StoreResult
- s3_SignExtendLoad
- s3_LoadAmt
- s3_DmemWr
- s_DmemData

For the **WB** pipeline stage, we will need the following datapath values and control signals:

- s3_StoreResult
- s4_MemtoReg
- s_RegWr
- s4_JALSelect
- s2_Load
- s5_PC+4

- s4_RegDest
- s5_RegInst
- s_Result
- s_Ovf
- s_Halt

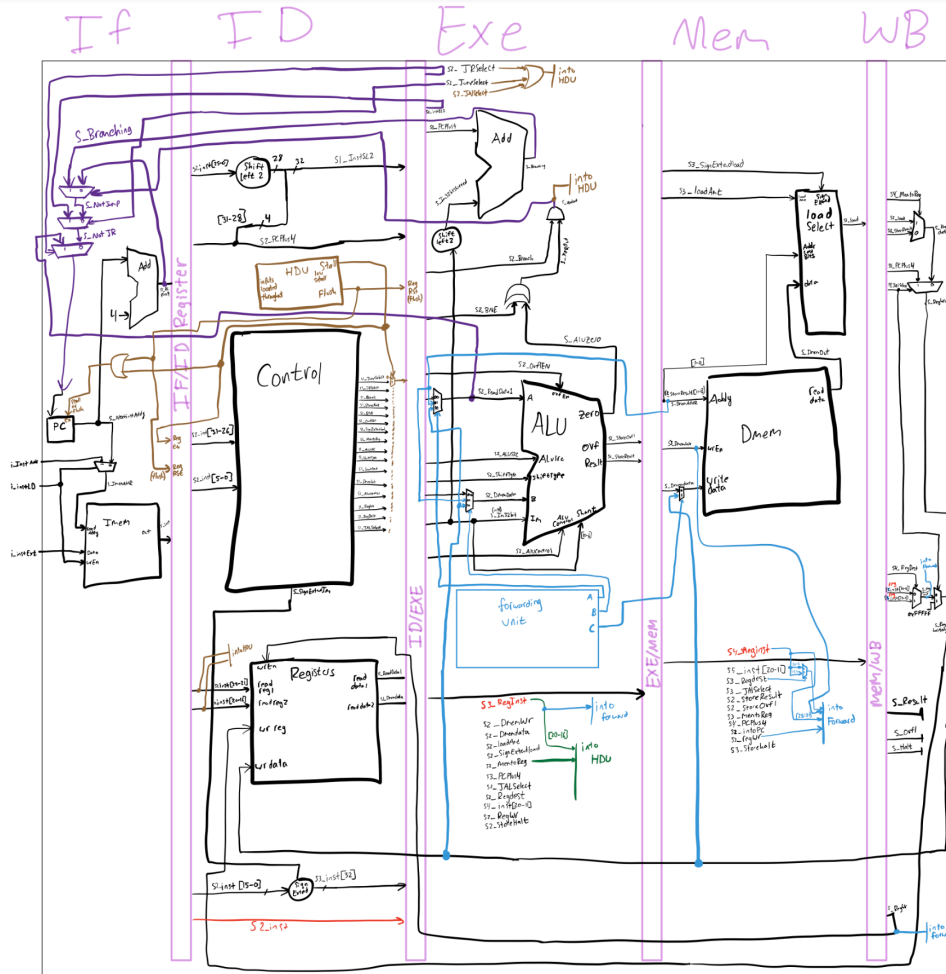
[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

The instructions that will cause a control hazard will be the instructions that update the PC and change the control flow of the pipeline. These instructions will be: **j, jr, jal, beq, bne**

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

In our hardware-scheduled pipeline, the execute stage performs the logic to update the PC register for branching and jumping. If a branch is taken or a jump is taken, this would result in the **Fetch stage and Decode stage** having incorrect instructions. Thus, **two flushes** will need to be executed for these stages whenever a branch (that is taken) or jump instruction occurs. Then the **Execute, Memory, and Writeback stages** will **need to be stalled** while the first two stages are being flushed.

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.

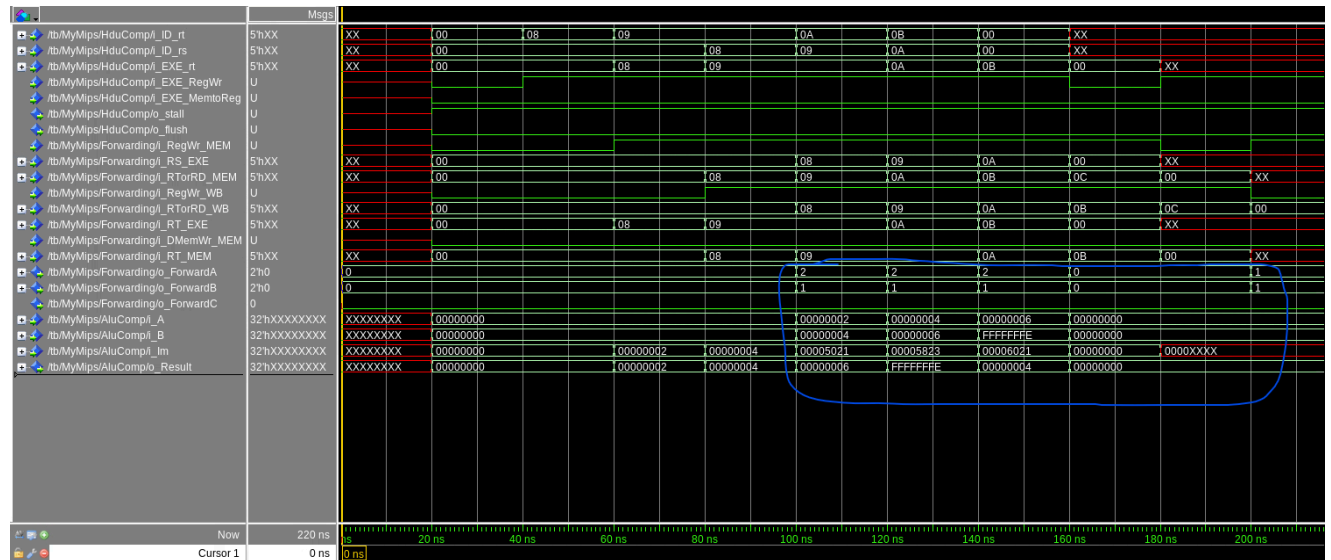


Note - We will include this as a PDF in submission. We know it is hard to read small writing on the google docs, and will include it in the submission as a PDF itself incase it is needed.

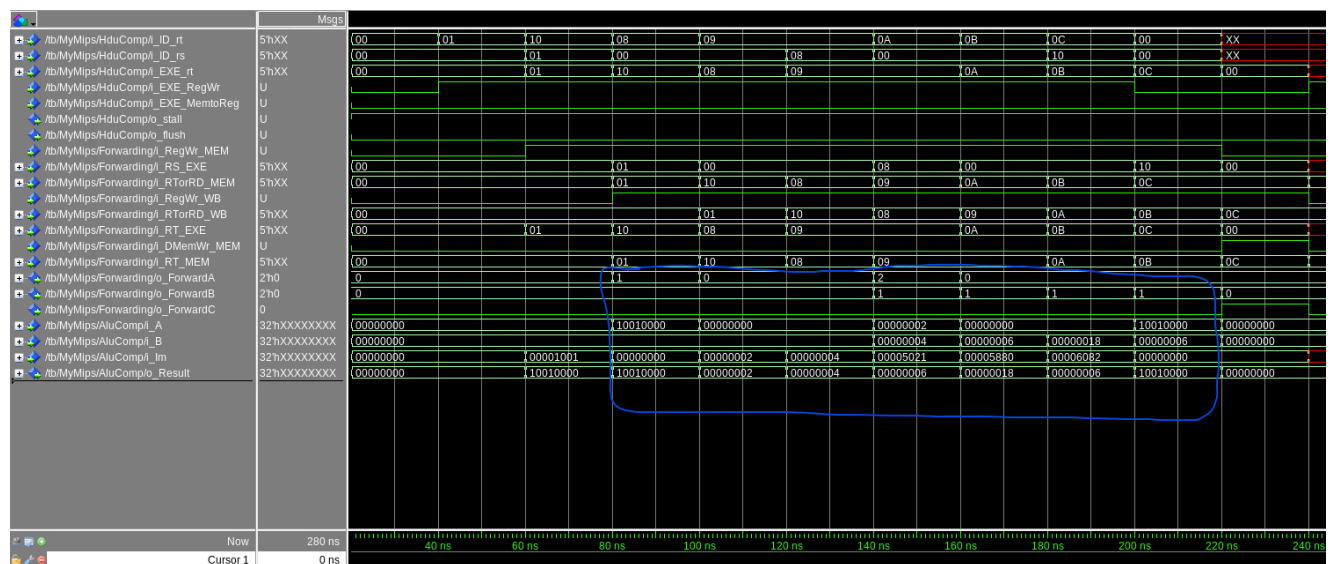
[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

Forwarding test cases:

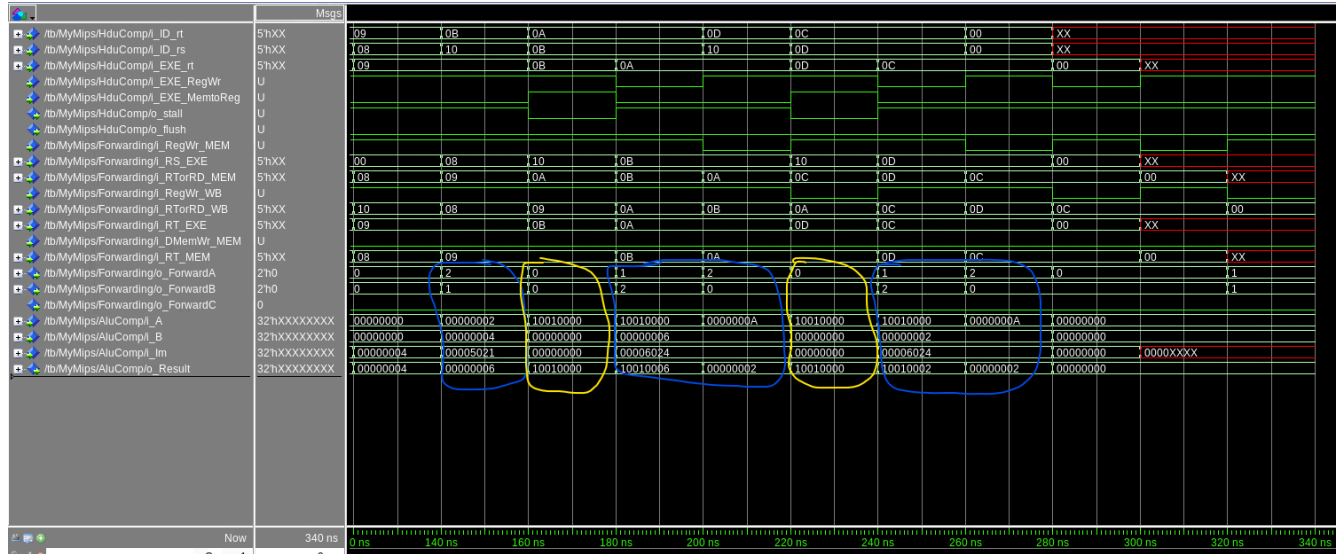
For our first forwarding test case in the *Proj2_Forwarding_case1.s* test file, we test that if there is a data hazard when the one arithmetic instruction tries to read the Rs register before its been written to Rs, then the final Rs value will be forwarded from input of the memory stage to input A of the ALU in the execution stage. In our test file for this case, we had four hazards that needed to be resolved by forwarding. The image below demonstrates the forwarding unit working and this test passing (indicated by the blue circle).



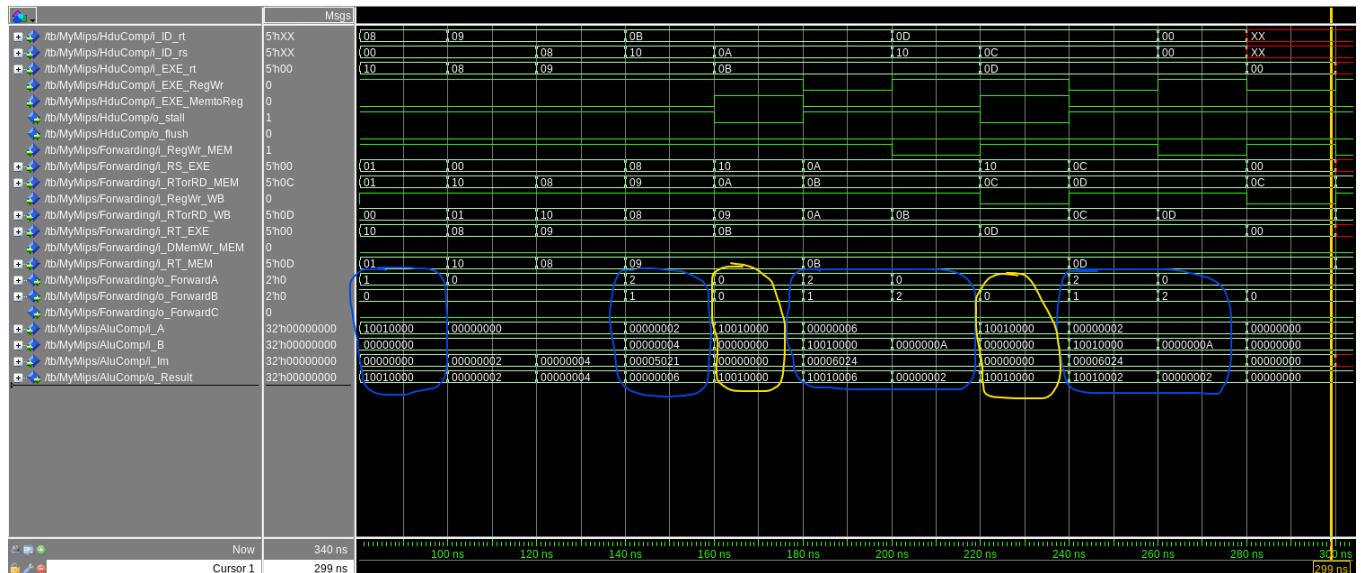
For our second forwarding test case in the *Proj2_Forwarding_case2.s* test file, we test that if there is a data hazard when the one arithmetic instruction tries to read the Rt register before its been written to Rt, then the final Rt value will be forwarded from input of the memory stage to input B of the ALU in the execution stage. In our test file for this case, we had five hazards that needed to be resolved by forwarding. The image below demonstrates the forwarding unit working and this test passing (indicated by the blue circle).



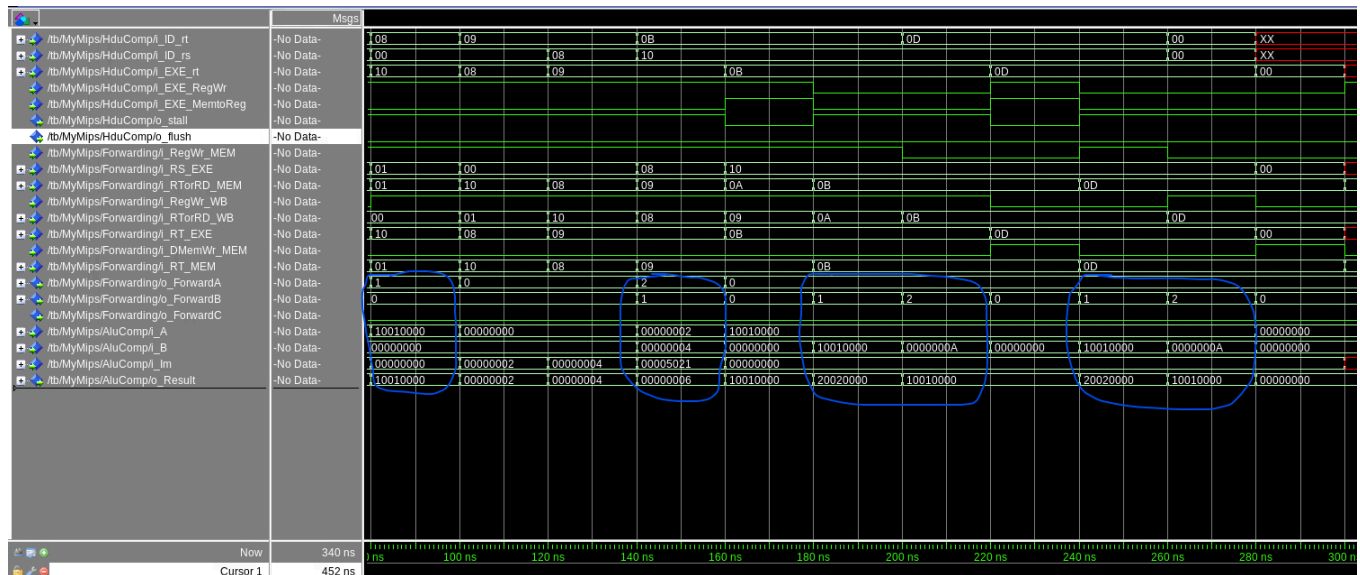
For our third forwarding test case in the *Proj2_Forwarding_case3.s* test file, we test that if there is a load-use data hazard when one instruction tries to read the Rs register before its been written to Rs, then the final Rs value will be forwarded from input of the writeback stage to input A of the ALU in the execution stage after one stall. In our test file for this case, we had four hazards that needed to be resolved by forwarding. The image below demonstrates the forwarding unit working and this test passing (indicated by the blue circle) while also stalling to get the final Rs value in the writeback stage to then forward (indicated by the yellow circle).



For our fourth forwarding test case in the *Proj2_Forwarding_case4.s* test file, we test that if there is a load-use data hazard when one instruction tries to read the Rt register before its been written to Rt, then the final Rt value will be forwarded from input of the writeback stage to input B of the ALU in the execution stage after one stall. In our test file for this case, we had four hazards that needed to be resolved by forwarding. The image below demonstrates the forwarding unit working and this test passing (indicated by the blue circle) while also stalling to get the final Rt value in the writeback stage to then forward (indicated by the yellow circle).

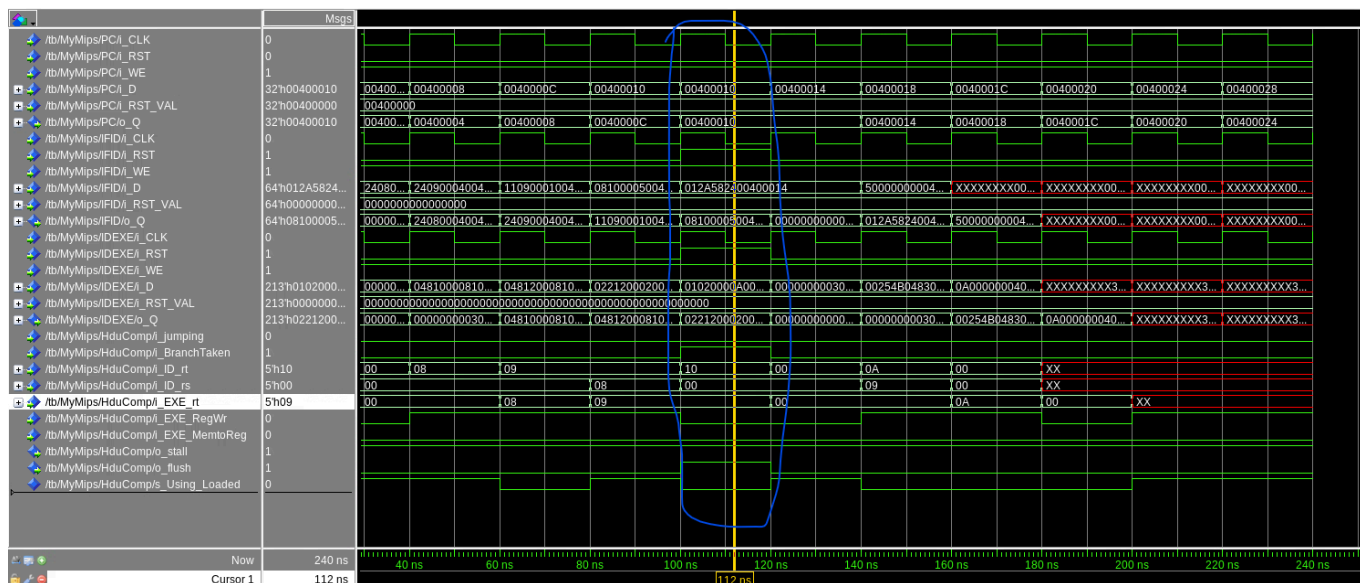


For our fifth forwarding test case in the *Proj2_Forwarding_case5.s* test file, we test that if there is a data hazard when one instruction (primarily the sw instruction) tries to read the a register before its been written in the writeback stage, then the final register value will be forwarded from input of the writeback stage to the dataIn input in the memory stage. In our test file for this case, we had six hazards that needed to be resolved by forwarding. The image below demonstrates the forwarding unit working and this test passing (indicated by the blue circle).

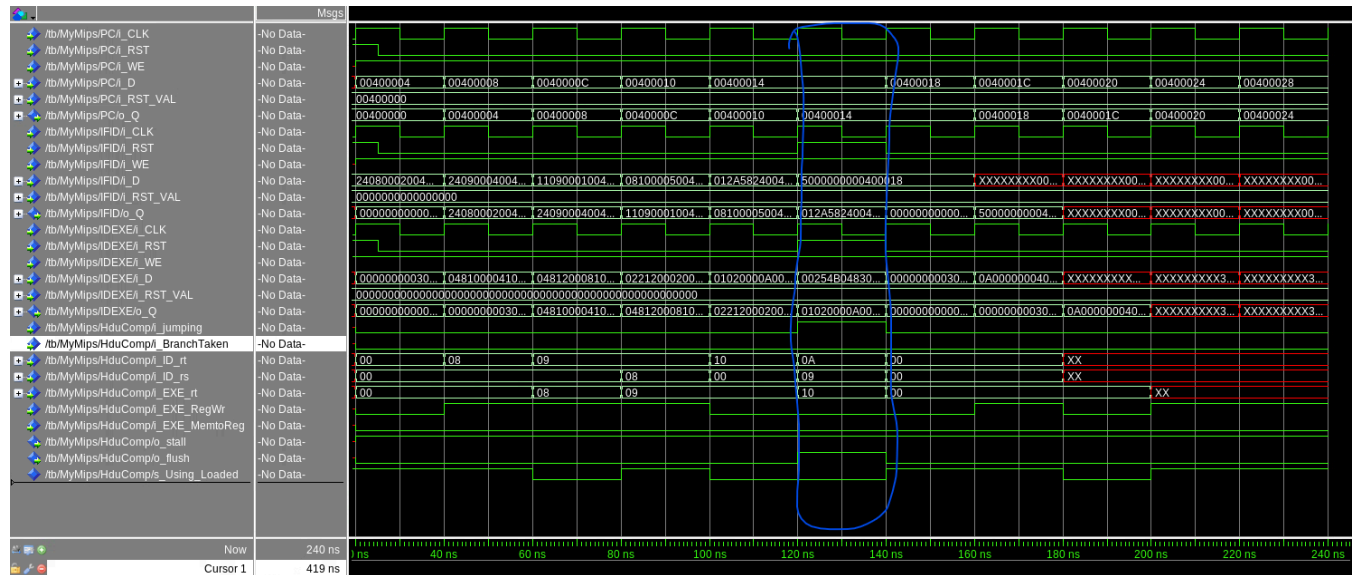


Control hazard avoidance test cases:

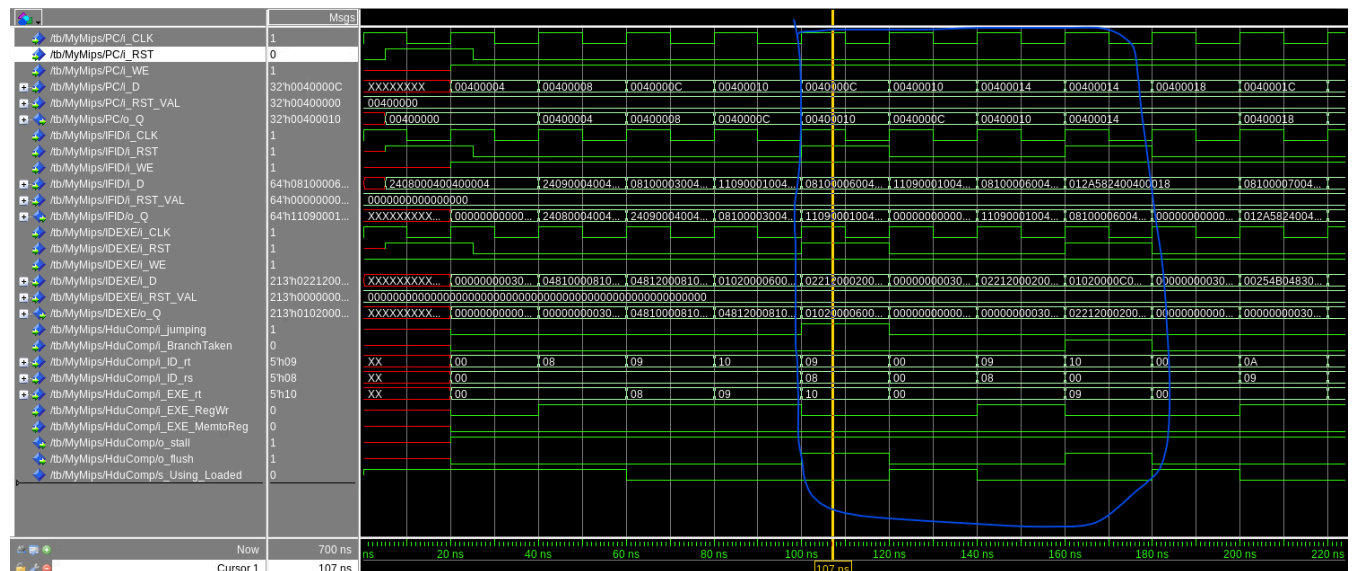
For our first control hazard avoidance test case in the *Proj2_ControlHazard_case1.s* test file, we test that if a branch (beq or bne) is taken, then a flush will occur in the pipeline to flush out the incorrectly fetched instruction in the PC. In our test file for this case, we had one control hazard that occurred because a beq instruction was taken/branched. The image below demonstrates our control hazard avoidance logic (indicated by the blue circle).

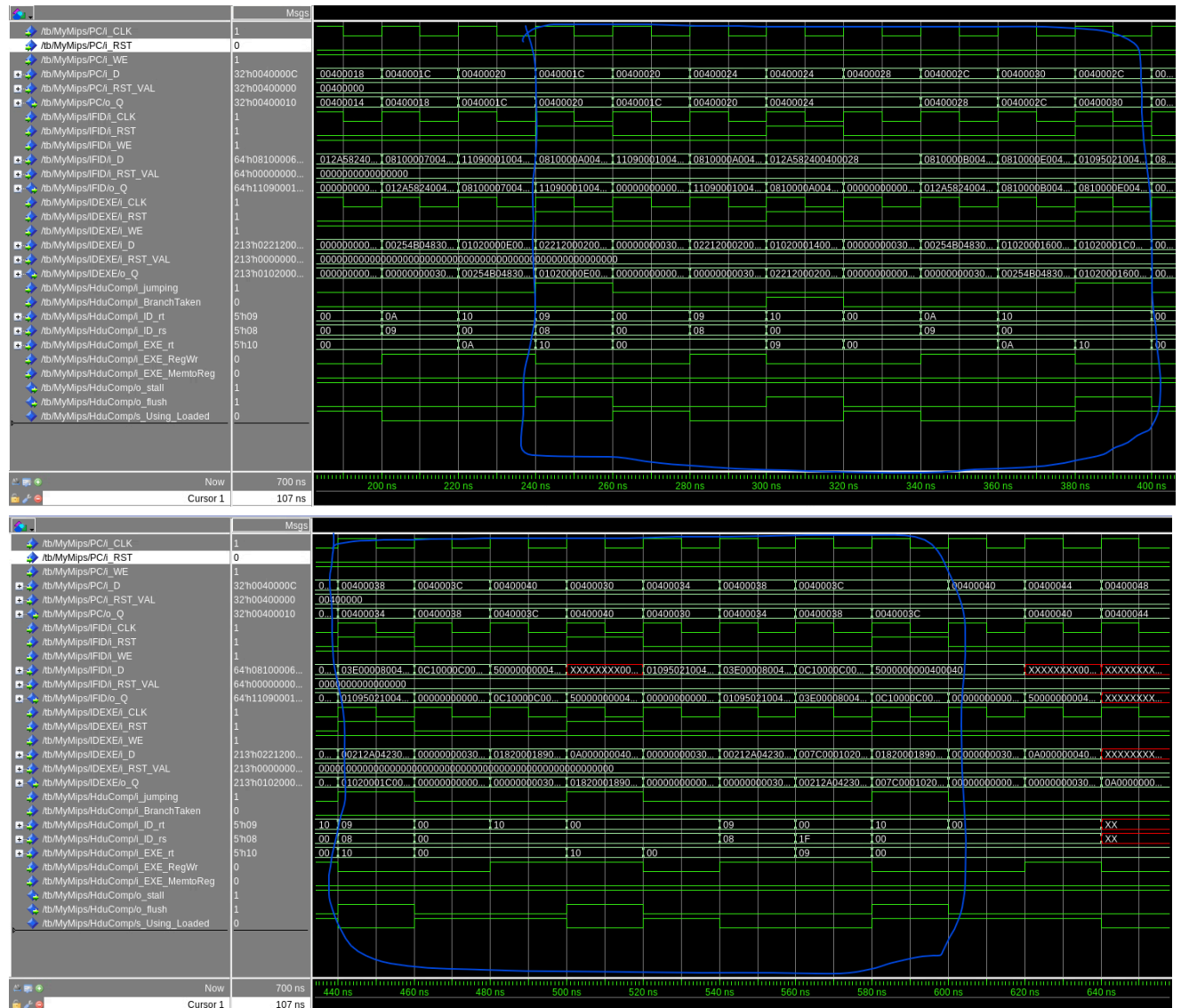


For our second control hazard avoidance test case in the *Proj2_ControlHazard_case2.s* test file, we test that if a branch (beq or bne) is not taken, then a flush will not occur in the pipeline. In our test file for this case, we had one control hazard that occurred because a beq instruction was not taken/branched, so a jump instruction occurs and causes a hazard that requires a flush. The image below demonstrates our control hazard avoidance logic (indicated by the blue circle).



For our third control hazard avoidance test case in the *Proj2_ControlHazard_case3.s* test file, we test that if a jump instruction (j, jal, jr) occurs, then a flush will occur in the pipeline. In our test file for this case, we had three control hazards that occurred, one for each jump instruction. This resulted in three total flushes being performed throughout the test file execution. The image below demonstrates our control hazard avoidance logic (indicated by the blue circle).





[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Forwarding and data hazard cases	# of occurrences in our testing files
Forwarding from input of the memory stage to input A of the ALU in the execution stage	9
Forwarding from input of the memory stage to input B of the ALU in the execution stage	12
Forwarding from input of the writeback stage to input A of the ALU in the execution stage after one stall	4

Forwarding from input of the writeback stage to input B of the ALU in the execution stage after one stall	4
Forwarding from input of the writeback stage to the dataIn input in the memory stage	4

For each forwarding and data hazard detection case, we made sure to have 3+ instances of each case within our test files. That way, there were plenty of chances to test all forwarding paths and possibilities that can occur within our pipeline design.

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Control hazard avoidance cases	# of occurrences in our testing files
A branch is taken	3
A branch is not taken	3
A jump occurs, causing flushing to occur	15

For each control hazard detection case, we made sure to have 3+ instances of each case within our test files. That way, there were plenty of chances to test that hazard detection unit worked and knew when to signal for flushing to occur in the pipeline registers.

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

The maximum frequency that our Hardware-scheduled pipelined processor can run at is **41.05 MHz**.

Our critical Path Starts in the MEMWB Register, and will go between a mux to select if we are writing to RT or RS. From there we are going to enter the forwarding unit. Then we will end up on the forwardB signal and go into the ALU as the “B” ALU value. From here to the adder subtractor and to the ALU Control mux. Then the zero flag output up through the XOR and AND gate. This will then feed into our HDU and end up as being the register RST signal for the IFID Register, where our critical path ends. This is our new critical path due to forwarding, and makes sense as there is a lot to go through. This is similar to the software one, which is what we expect given we did not significantly change the design.

